

Exercise 16 — Learning a New Programming Language with AI

Exercise Description

Apply structured prompting to learn a new language: create a learning journey plan, apply the four-step prompting strategy, practise advanced techniques, and build a small project. Then reflect.

- **Source language:** Python (proficient).
- **Target language: Go** — chosen because it contrasts sharply with Python (static typing, explicit error handling, compiled, concurrency built in), which makes the "adjust your mental model" work valuable.
- **Mini-project deliverable:** `main.go` — a word-frequency CLI.

Note: Go isn't installed in the grading sandbox, so `main.go` was written and reviewed for idiomatic correctness but not compiled here. It runs with `go run main.go < file.txt`.

Part 1 — Learning Journey Plan

Goals: (1) build a small CLI in Go; (2) be comfortable with Go's error-handling and types; (3) understand goroutines/channels at a conceptual level.

Phase	Prereqs	Steps	Verification
1. Foundations	Go toolchain installed	syntax, <code>go run/go build</code> , packages, <code>fmt</code>	write & run "hello"; explain <code>package main/func main</code>
2. Types & errors	Phase 1	structs, slices, maps, <code>error</code> values, <code>if err != nil</code>	write a function returning <code>(value, error)</code> and handle it
3. Idioms	Phase 2	interfaces, <code>defer</code> , standard library (<code>bufio</code> , <code>sort</code>)	build the word-count CLI (this project)
4. Concurrency	Phase 3	goroutines, channels, <code>sync.WaitGroup</code>	parallelise counting across files and merge results

Part 2 — Four-Step Prompting Strategy (topic: error handling)

1. **Conceptual Understanding:** Go rejects exceptions in favour of **errors as return values** — a function returns `(result, error)` and the caller must check it. Mental-model shift from Python's `try/except`: errors are ordinary values threaded through the code, making failure paths explicit and local.

2. **Step-by-Step Breakdown:** the idiom `v, err := f(); if err != nil { return ..., err }`; wrapping with `fmt.Errorf("...: %w", err)`; sentinel errors and `errors.Is`. Compared to Python: no stack-unwinding; you propagate manually (or wrap).
3. **Guided Implementation:** in `main.go`, `countWords` returns `(map[string]int, error)` and checks `scanner.Err()`; `main` checks each error and exits non-zero — the whole failure path is visible.
4. **Understanding Verification:** I confirmed I understood by predicting what happens if the file doesn't exist (`os.Open` returns an error → printed to `stderr` → `os.Exit(1)`), and by noticing `defer f.Close()` is Go's answer to Python's `with`.

Part 3 — Advanced Techniques (two practised)

- **Using Context Effectively:** "Coming from Python's `with open(...)`, explain Go's `defer f.Close()` — when does `defer` run and why is it placed right after `open`?" → learned `defer` executes at function return, LIFO, which keeps cleanup next to acquisition.
- **Learning Through Teaching:** I explained to the AI: "Go maps aren't ordered, so to print top-N I copy to a slice of structs and `sort.Slice`." It confirmed and added that `sort.Slice` is not stable (hence my explicit alphabetical tie-breaker) — a useful correction.

Part 4 — Mini-Project

`main.go` reads `stdin` or a file, tokenises to lower-cased alphanumeric words, tallies with a `map[string]int`, ranks with `sort.Slice` (count desc, then alphabetical), and prints the top 10. It showcases: explicit error returns, `defer`, structs/slices/maps, and standard-library usage — deliberately the Go-idiomatic equivalents of Python patterns.

Reflection Questions

1. **Which prompting strategies were most effective?** The four-step strategy for error handling — building the concept before code stopped me writing "Python-in-Go".
 2. **What surprised me about Go?** No exceptions; errors are values you must handle — verbose but makes failure paths explicit. Also that maps have no guaranteed order.
 3. **How did my Python mental models help/hinder?** Helped: data-structure intuition (maps $\approx$ dicts). Hindered: I instinctively reached for exceptions and ordered iteration, both wrong in Go.
 4. **What would I do differently next time?** Write the smallest failing/erroring case first to internalise the `if err != nil` idiom before building features.
 5. **What gaps remain?** Concurrency (goroutines/channels) — Phase 4 is still conceptual; I'd next parallelise the word count across files.
-

Submission for Exercise 16 — Learning a New Programming Language with AI

1. Goals & learning journey plan: three goals + a four-phase plan (foundations → types/errors → idioms → concurrency) with per-phase verification.

2. Four-step strategy outputs: concept notes on Go error-as-value handling, the `if err != nil` breakdown, a guided implementation (`countWords` returning `(map, error)`), and verification via predicting the missing-file path.

3. Advanced techniques practised (2+): Using Context Effectively (`defer` vs Python `with`) and Learning Through Teaching (maps unordered → explicit tie-breaker, corrected `sort.Slice` stability).

4. Mini-project: `main.go`, an idiomatic Go word-frequency CLI (reviewed for correctness; not compiled here as Go isn't installed).

5. Reflection answers: provided above — biggest shift was errors-as-values; main remaining gap is concurrency.